

懂你心情的「線上盲盒書店助理」

文學情感共鳴 RAG 推薦系統 —— MVP 產品需求文件 (PRD)

文件版本：v1.0 | 目標讀者：獨立開發者、AI 策略 PM、Cursor/Codex AI 助手

一、MVP 核心定位與驗證目標

本 MVP（最小可行性產品）的核心目的是用最快的速度、最低的成本，在本地搭建一個具備網頁視覺交互的「懂心情的文學推薦助手」。我們希望透過這個 MVP 驗證兩件事：

- **情感理解能力**：AI 能不能準確地把用戶「口語化、發洩式」的情緒，轉化為準確的文學意象與找書密碼，而不是傻傻地進行字面匹配。
- **交互體驗價值**：用戶在看到 AI 特製的「手寫小紙條（推薦理由）」後，是否真的能產生「被治癒、被說中」的情感共鳴。

MVP 開發原則：快、穩、省

拒絕複雜的雲端配置，拒絕高昂的付費數據庫。全數據本地化存儲，前端使用極簡網頁框架（如 Streamlit 或 Gradio），保證在面試或展示現場「秒級響應、絕對不掉鏈子」。

二、MVP 用戶旅程與畫面設計 (UI/UX)

為了保持極簡，MVP 的網頁只需要 1 個單頁畫面，包含三個核心視覺區塊：

1. 頂部：心境輸入框

- 一個大文本框，提示詞為：「留下你此時此刻的心情、發呆的念頭，或是想看到的畫面...」
- 一個按鈕：「翻閱專屬書架」。

2. 中部：AI 思考狀態欄（展示策略邏輯，面試必備高光）

- 當用戶點擊按鈕後，頁面動態展示小助手的思考日誌（例如：「正在讀懂你的弦外之音...」->「正在書架上為你淘書...」->「正在為你手寫推薦紙條...」）。

3. 底部：精美選段卡片（最終交付物）

- 並排或垂直蹦出 3 張精美的文學卡片。每張卡片包含：【書名】、【作者】、【經典原文段落（300-500字）】、以及一封溫暖的【助理手寫小紙條（推薦理由）】。

三、核心功能需求（三大基礎積木）

F1. 私人電子書架與本地存儲 (Local ChromaDB)

- **功能描述**：在本地建立一個普通資料夾 `data/raw/`，放入 5-10 本自己喜歡的經典文學/散文電子書（.txt 格式）。

- **處理邏輯**：寫一段簡單的腳本，將文本切成 400 字左右的小方塊。在存入本地 ChromaDB（生成 chroma.sqlite3 檔案）前，調用大模型為每段話貼上標籤（Metadata），包含：作家名、書名、包含的意象（如：下雨、黃昏）、展現的情緒底色（如：孤獨、平靜）。

F2. 智能體核心流水線 (LangGraph Core Pipeline)

在後台使用 LangGraph 連接 4 個核心助手小幫手，數據在它們之間流轉：

小幫手節點	核心職責（大白話描述）	狀態更新內容
1. 翻譯官 (Intent Node)	拒絕字面匹配。把用戶的話翻譯成 3 個用來找書的「秘密暗號」（擴充檢索詞）。	analyzed_intent (包含情緒與意象標籤)
2. 圖書管理員 (RAG Node)	拿著暗號去本地 ChromaDB 檔案裡，依照「靈魂密碼（向量）」的比對，抱出 10 個候選段落。	retrieved_docs (10 個原始片段)
3. 質檢員 (Review Node)	策略核心： ① 檢查分數，低於 60 分直接打回讓「翻譯官」重新想（上限 2 次）。 ② 多樣性過濾：如果前 3 名都是同一個作家，強行拿走換人，保證花樣。	filtered_docs (挑選出的完美 3 個片段)
4. 電台主持人 (Output Node)	用溫柔、優雅且克制的語氣，針對這 3 段文字手寫「推薦紙條」，解釋這段話如何精準擊中用戶此時的心境。	output_response (網頁最終顯示的文本)

F3. 安全與防禦機制 (Guardrails)

- **防死循環**：質檢員打回重試的次數嚴格限制為 2 次，若 2 次都不及格，走向【溫柔兜底節點】，主動跟用戶說：「我仔細翻了書架暫時沒找到完美的，要不要換個說法，加點‘下雨天’或‘舊書攤’的畫面試試？」
- **防胡言亂語**：最終輸出時，嚴禁 AI 隨意修改世界名著的原文，必須與數據庫中的 doc_id 原文完全一致。

四、4 週 MVP 落地執行計劃與任務清單

第 1 週：冷啟動與本地書架搭建（搞定數據）

- **任務 1-1**：下載 5 本具備強烈文藝感、情緒底色明顯的文學作品（如村上春樹、太宰治、經典散文集），保存為 txt。
- **任務 1-2**：編寫 ingest.py 腳本，用代碼把 txt 切成 400 字一段的切片。
- **任務 1-3**：呼叫低成本大模型（如 GPT-4o-mini），批量給這幾百個切片打上結構化標籤（作者、意象、情緒）。
- **任務 1-4**：把切片、標籤和向量數據一起存入本地 ChromaDB，確認本地生成了數據庫資料夾。

第 2 週：後台 Agent 流水線編排（搞定大腦）

- **任務 2-1**：在 `nodes.py` 中寫好「翻譯官」、「圖書管理員」、「質檢員」、「電台主持人」4 個 Python 函數。
- **任務 2-2**：調教提示詞（Prompt Engineering），確保「翻譯官」懂隱喻，「電台主持人」說話有溫度、不油膩。
- **任務 2-3**：使用 LangGraph 把這 4 個函數連線起來，寫好分叉路口（大於 60 分去輸出，小於 60 分去重試）。
- **任務 2-4**：寫一個本地的 Python 去重邏輯，實現「同一個作者在一次推薦中不能出現 2 次以上」的打散策略。

第 3 週：網頁界面開發與前後端聯調（搞定衣服）

- **任務 3-1**：選擇 Streamlit（推薦，只需幾十行 Python 即可生成網頁）或 Gradio 搭建單頁交互畫面。
- **任務 3-2**：設計輸入框、按鈕，並把後台的 LangGraph 狀態機接口接到網頁按鈕上。
- **任務 3-3**：優化視覺，把 3 個文學推薦結果排版成像「深夜樹洞小卡片」一樣的精美區塊。
- **任務 3-4**：在網頁上把 AI 的思考日誌（Log）實時打印出來，增強「Agent 在幫我思考」的感知度。

第 4 週：邊界測試、優化與現場演示準備（搞定面試）

- **任務 4-1**：輸入極端心情（如：“我想發瘋”、“哈哈哈哈哈高興”、“...”），測試兜底功能是否會溫柔拒絕，確保系統不崩潰。
- **任務 4-2**：檢查有無幻覺，確保書名、作者、原文沒有串線或被 AI 自行編造。
- **任務 4-3**：錄製一個 2 分鐘的本地運行演示視頻（Screen Share 錄屏），作為雙保險，防止面試現場斷網或大模型 API 響應過慢。

五、MVP 成功度量指標 (Metrics)

既然是展示用的 MVP，我們在現場展示或小範圍公測時，主要關注以下 2 個指標：

1. **現場首發滿意度 (Wow Factor)**：面試官或體驗者輸入自己的真實心情後，看到推薦語和段落時，是否露出驚訝、讚許的神情（主觀指標）。
2. **策略可控率 (Technical Metric)**：檢查後台日誌，看質檢員（Review Node）的觸發是否準確，打散策略是否成功阻斷了單一作者的壟斷，重試機制是否在 2 次內收斂。